ARSD COLLEGE, UNIVERSITY OF DELHI
B.Sc. (Hons.) Computer Science, Semester V
DSC 15: Software Engineering

AIDLY

A Venue Operations Layer for Crowd Emergencies

Software Engineering Theory Assignment

First Draft

Team Member	Roll Number
Janvi Monga	24/48044
Sourav Kumar	24/48092
Avijit Sakkarwal	24/48079

Batch 2026 to 2027
Submitted for evaluation, 26 marks, team wise

Contents

1	**Problem Statement**	**4**
1.1	Overview	4
1.2	Challenges in the Current System	4
1.3	The Proposed Solution	5
1.4	Scope of the System	5
2	**Requirements Identification**	**6**
2.1	Method: A Throwaway Prototype	6
2.2	The Prototype	6
2.3	Findings from the Prototype	7
2.4	Functional Requirements	7
2.5	Nonfunctional Requirements	8
2.6	Requirements Traceability	8
3	**Process Model**	**10**
3.1	Candidate Models from the Syllabus	10
3.2	Comparison Against Project Characteristics	10
3.3	Selected Model: The Incremental Model	10
3.4	Justification	11
3.5	Why the Other Models Were Rejected	11
3.6	Planned Increments	12
4	**Data Flow Diagrams and Data Dictionary**	**13**
4.1	Notation Used	13
4.2	Level 0: Context Diagram	13
4.3	Level 1 Data Flow Diagram	15
4.4	Data Dictionary	17
5	**Use Case Diagram**	**19**
5.1	Actors	19
5.2	The Diagram	20
5.3	Use Case Inventory	21
6	**Use Case Templates**	**22**
6.1	UC 01: Submit Emergency Report	22
6.2	UC 02: Dispatch Responder	24
6.3	UC 03: Authenticate Control Room Operator	25
7	**Login Module Connection with Database**	**26**

7.1	Overview	26
7.2	Database Schema	26
7.3	Connection Layer	27
7.4	Password Hashing and Tokens	27
7.5	Authentication Service	28
7.6	The Login Endpoint	29
7.7	Execution Transcript	29
7.8	Outcome	30
8	**Conclusion and Next Steps**	**31**

1
Problem Statement

1.1 Overview

Large public gatherings are ordinary events in India. A college festival, a stadium fixture, a religious procession or a concert can place tens of thousands of people inside a single boundary for several hours. Every one of those venues already employs the people needed to handle an emergency: marshals, medical staff, fire wardens and a control room that supervises them. What no venue reliably has is a channel that carries what an attendee can see to the people who are paid to act on it.

When something goes wrong inside a dense crowd, the accepted method of summoning help is a voice call to an emergency number. That method fails at exactly the moment it is needed most. The failure is not occasional. It is structural, and it has three separate causes that all appear together in the same conditions.

1.2 Challenges in the Current System

1. **Speech becomes unusable.** A crowd generates continuous ambient noise. The caller cannot hear the operator and the operator cannot hear the caller. The information that matters, namely what has happened and where, is the information least likely to survive the call.
2. **The cellular network saturates.** Thousands of handsets contend for one cell tower. Voice calls are dropped or refused precisely when the largest number of people are trying to place them. A short structured message of a few hundred bytes will frequently succeed where a call will not.
3. **One call carries one view, once.** Thirty people witnessing a single crush produce thirty separate conversations rather than one prioritised incident. Nobody in the chain sees that the reports describe the same event, so the scale of the event is invisible at the moment it must be judged.
4. **The venue is not the recipient.** India operates the Emergency Response Support System, reached on 112, and it handles the path from a citizen to the state well. It routes that citizen to a state control room. It does not give one particular venue a live picture of its own ground, it does not dispatch that venue's own marshals and medics, and it cannot push exit guidance to the people standing inside that venue.
5. **Some reports cannot be made openly.** Harassment at crowded events is common and heavily under reported. A loud or visibly obvious alert can increase the risk to the person reporting rather than reduce it, so any system that only offers a conspicuous alarm will simply not be used by the people who most need it.

Taken together these produce a gap that is easy to state. At a festival or a stadium, trained responders are frequently standing within a hundred metres of an incident, and there is no system that routes a report to them, resolves where that report came from, or tells the control room that thirty messages describe one event rather than thirty events.

1.3 The Proposed Solution

Aidly is a venue operations layer for crowd emergencies. Anyone inside a venue reports in a single tap, optionally attaching a short description or a photograph. The system resolves the report to a named zone of the venue, classifies it, merges it with other reports describing the same event, and presents the control room with a live and deduplicated picture of its own ground. The operator dispatches the venue's own responders, tracks acknowledgement and arrival, and broadcasts guidance to attendees where that is needed.

The design premise is that the network is unreliable exactly when the system matters. A tap is a few hundred bytes and gets through when a voice call will not. Where location services fail, the reporter chooses the nearest gate rather than being blocked from reporting at all.

Aidly does not replace 112. It is the layer that 112 does not provide, and it escalates to 112 rather than competing with it. This distinction is deliberate and it governs every requirement that follows.

1.4 Scope of the System

Included	Excluded
A single venue divided into named zones, configured by an administrator.	Any claim to replace or duplicate the national 112 service.
Four human actors: reporter, responder, control room operator and venue administrator.	Direct programmatic integration with the ERSS backend, for which no public interface is published.
Incident categories covering fire, medical, crowd, accident, violence, harassment and other.	Dedicated hardware such as wearables or fixed panic buttons.
A mobile first web application requiring no installation by an attendee.	Nationwide consumer deployment outside a venue that runs the system.
Live updates to operator and responder screens without polling.	Locating a handset through the telecom carrier.

Table 1.1 Scope boundary of the system

2
Requirements Identification

2.1 Method: A Throwaway Prototype

Requirements for this system could not be gathered by interview alone. The behaviour under discussion happens in conditions that are difficult to describe from memory, and the people who would use the reporting screen are ordinary attendees rather than trained operators. The team therefore built a working prototype first and used it as the instrument of elicitation, which is the approach Pressman describes as prototyping within the evolutionary process models.

The prototype is a throwaway in the strict sense. It was built to answer questions about the interface and the vocabulary, and its code is not intended to become the delivered product. What survives from it is the requirement set recorded in this chapter.

2.2 The Prototype

The prototype implements the reporter facing path end to end. It presents a single prominent action on the landing screen, a category chooser, a location control that degrades to a gate list when the browser cannot supply a position, an optional free text description with a character budget, and an optional callback number.

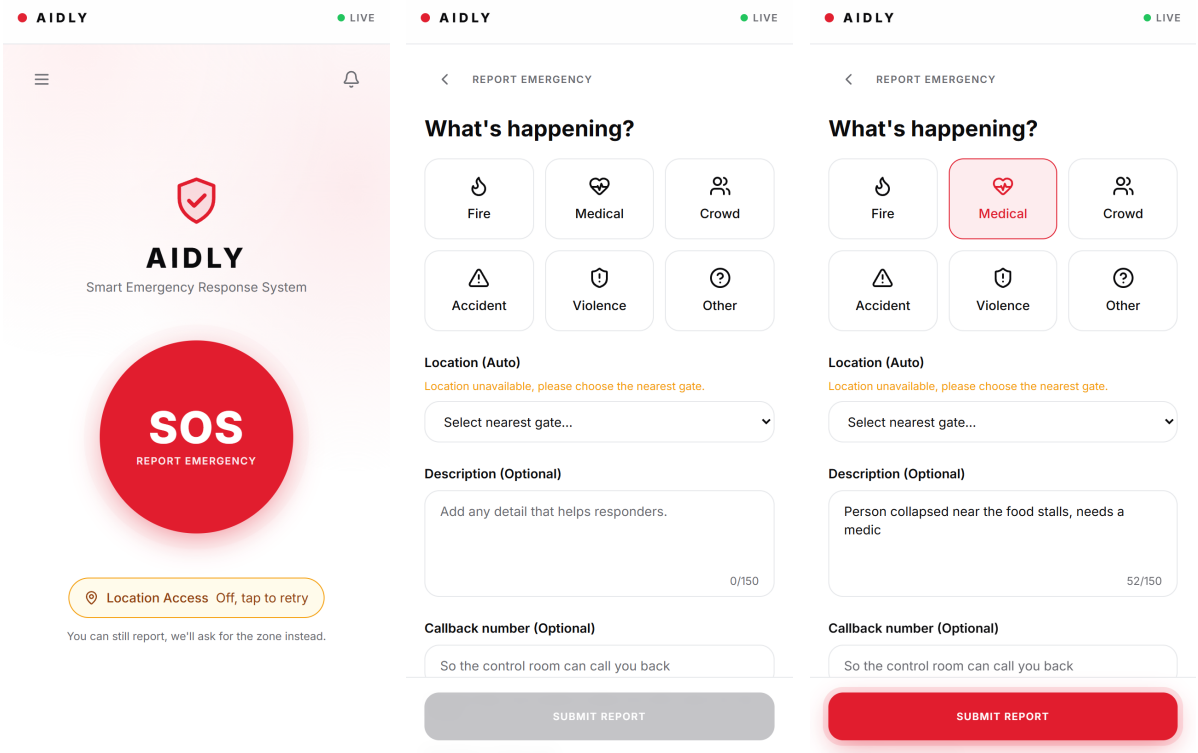


Figure 2.1 Prototype screens. Left: the landing screen with the single reporting action and the degraded location notice. Centre: category selection and the gate fallback. Right: a completed report ready for submission.

2.3 Findings from the Prototype

1. **One action must dominate the landing screen.** Early layouts offered several entry points and testers hesitated. A single large control removed the hesitation entirely.
2. **Location must never block a report.** The browser refused to supply a position in a large proportion of trials. Treating this as an error produced an unusable screen, so the prototype was changed to ask for the nearest gate instead, and the report proceeds.
3. **Six categories are sufficient and are understood.** Testers selected the intended category without instruction. A longer list slowed selection without improving accuracy.
4. **Free text must be optional and short.** A required description delayed submission. A visible character budget of 150 characters kept entries specific.
5. **The callback number needs an explanation.** Testers assumed the number would be used to locate them. Stating plainly that location comes from the device and never from the number removed the objection.

2.4 Functional Requirements

ID	Requirement	Source
FR 01	The system shall allow any attendee to submit a report in a single action without registration or login.	Prototype
FR 02	The system shall let the reporter classify the report into one of the defined categories.	Prototype
FR 03	The system shall resolve each report to a named zone, using the device position where available and a reporter chosen gate otherwise.	Prototype
FR 04	The system shall accept an optional description of at most 150 characters and an optional photograph.	Prototype
FR 05	The system shall accept an optional callback number and shall never use it to determine location.	Prototype
FR 06	The system shall assign a severity tier to every incident and shall record which method produced that tier.	Analysis
FR 07	The system shall merge reports of the same category, in the same zone, within a configured time window into one incident and shall maintain a corroboration count.	Analysis
FR 08	The system shall dispatch the nearest available responder whose role and attributes match the incident category.	Analysis
FR 09	The system shall escalate an unacknowledged high severity incident automatically after a configured interval.	Analysis
FR 10	The system shall provide a documented escalation handoff to ERSS 112 initiated by the control room.	Analysis
FR 11	The system shall allow a report of a personal safety incident to be submitted discreetly, producing no sound, no vibration and no visible change of screen state.	Analysis
FR 12	The system shall authenticate control room operators, responders and administrators before granting access to any operational screen.	Analysis

ID	Requirement	Source
FR 13	The system shall allow the control room to broadcast advisories and exit guidance to attendees.	Analysis
FR 14	The system shall maintain an append only audit record of every state change and every access to attached evidence.	Analysis

Table 2.1 Functional requirements

2.5 Nonfunctional Requirements

ID	Category	Requirement
NFR 01	Performance	A submitted report shall appear on the control room screen within two seconds under normal network conditions.
NFR 02	Reliability	The reporting screen shall remain usable when the geolocation service is unavailable.
NFR 03	Usability	An attendee with no prior exposure shall be able to submit a correctly categorised report without instruction.
NFR 04	Security	Passwords shall be stored only as salted one way hashes. Plain passwords shall never be written to storage or to any log.
NFR 05	Security	A failed login shall return the same message and take comparable time regardless of whether the account exists.
NFR 06	Privacy	The identity of a reporter shall be visible to the control room and shall not be exposed to responders who do not require it.
NFR 07	Portability	The whole system shall run on hardware the venue controls, with no dependence on any managed third party service.
NFR 08	Auditability	Every incident shall be reconstructible from the audit record alone.

Table 2.2 Nonfunctional requirements

2.6 Requirements Traceability

Each functional requirement is carried forward into the later chapters. The matrix below records where every requirement is realised, so that no requirement is left without a design and no part of the design exists without a requirement behind it.

Requirement	DFD process	Use case	Implemented in this draft
FR 01, FR 02, FR 03, FR 04, FR 05	1.0 Report Intake	UC 01	Prototype only
FR 06	2.0 Triage	UC 06	No, classifier pending
FR 07	3.0 Merge and Corroborate	UC 06	No
FR 08	4.0 Dispatch	UC 02	No
FR 09	5.0 Escalation	UC 07	No

Requirement	DFD process	Use case	Implemented in this draft
FR 10	5.0 Escalation	UC 08	No, specified only
FR 11	1.0 Report Intake	UC 04	No
FR 12	1.0 Report Intake, 4.0 Dispatch	UC 03	Yes, Chapter 7
FR 13	6.0 Broadcast	UC 09	No
FR 14	7.0 Audit and Evidence	UC 08	No

Table 2.3 Requirements traceability matrix

3
Process Model

3.1 Candidate Models from the Syllabus

Five process models are considered, all of them drawn from the course syllabus: the Waterfall model, the Prototyping model, the Spiral model, the Agile approach represented by Scrum, and the Incremental model.

3.2 Comparison Against Project Characteristics

The choice of model follows from the characteristics of this particular project rather than from a general preference. Four characteristics dominate.

- **The core requirements are already known.** A prototype has been built and evaluated, and Chapter 2 records a stable requirement set.
- **The system must be demonstrable at every stage.** This is an assessed academic project with fixed review points, so a model that produces nothing runnable until the end carries unacceptable risk.
- **The functions are separable.** Reporting, triage, dispatch, escalation and broadcast are distinct and can be delivered one after another, each leaving a working system.
- **The team is small and co located.** Three members, without a dedicated customer representative available on demand.

Model	Fit	Deciding factor
Waterfall	Poor	Assumes requirements are frozen and delivers nothing executable until late. Any change discovered during construction is expensive to absorb.
Prototyping	Partial	Already used, and used correctly, to elicit requirements. It is a technique within the analysis phase here rather than the model for the whole build.
Spiral	Poor	Its strength is repeated formal risk analysis, which suits large and costly projects. The overhead is disproportionate to a project of this size and duration.
Scrum	Partial	Attractive for its iteration, but it presumes a product owner available continuously and it de emphasises the fixed documentation set this submission must produce.
Incremental	Strong	Delivers the system in functional increments, each one testable and demonstrable, while retaining the documented analysis and design stages the course requires.

Table 3.1 Process model comparison

3.3 Selected Model: The Incremental Model

The **Incremental model** is selected. In this model the requirements are established once, then partitioned into increments. Each increment passes through analysis, design, coding and testing, and each delivers a working addition to the product. The first increment produces a usable core, and every increment after it extends that core without breaking it.

3.4 Justification

1. **The requirements are stable enough to partition.** The prototype resolved the questions that would otherwise have forced continuous renegotiation, so the requirement set can be fixed and then divided. This is the precondition the Incremental model assumes.
2. **Every increment leaves a working system.** This matters more here than in an ordinary project. An emergency system that is half built is not partially useful, it is dangerous, so each increment is defined to end at a point where the whole path from report to resolution still functions.
3. **Risk is reduced by ordering.** The first increment carries the highest technical risk, namely the database, the schema and authentication. Placing it first means the riskiest work is proven while there is still time to respond to what it reveals.
4. **It matches the assessment structure.** The course requires a documented analysis and design stage. The Incremental model preserves those stages rather than replacing them with continuous negotiation.
5. **It suits a team of three.** Increments can be assigned as whole vertical slices, so each member owns a complete function rather than a fragment of one.

3.5 Why the Other Models Were Rejected

3.5.1 Waterfall

The Waterfall model requires that requirements be complete and frozen before design begins, and it produces no executable output until construction is well advanced. This project has already changed direction once in response to external feedback. A model with no cheap route for absorbing such a change is the wrong instrument, and the absence of an early running system would leave defects undiscovered until late.

3.5.2 Spiral

The Spiral model is built around repeated risk analysis at every cycle. That machinery earns its cost on large, expensive and long running systems. On a project of this size the risk analysis effort would exceed the construction effort, and the identified risks are few enough to be handled by the ordering of increments described above.

3.5.3 Agile and Scrum

Scrum was the closest competitor. It was rejected for two reasons. It assumes a product owner who is continuously available to reprioritise a backlog, and no such role exists on this team. It also values working software above comprehensive documentation, whereas this submission is assessed substantially on its documentation. The Incremental model provides the iteration that

made Scrum attractive without discarding the documentation the course requires.

3.5.4 Prototyping as the whole model

Prototyping has already served its purpose in Chapter 2. Retaining it as the model for the entire build would risk the classic failure of that approach, in which an exploratory prototype is progressively patched into a production system and carries its shortcuts forward. The prototype here is explicitly a throwaway.

3.6 Planned Increments

Increment	Delivered function	Leaves working
I	Database schema, connection layer and authentication for operators, responders and administrators.	A system that a real user can log into.
II	Report intake, zone resolution and persistence.	An attendee can file a report that is stored and retrievable.
III	Triage, deduplication and the control room incident view.	The control room sees merged incidents with corroboration.
IV	Dispatch, acknowledgement, resolution and escalation.	The full path from report to resolution operates.
V	Broadcast, discreet reporting and the audit and evidence record.	The complete requirement set of Chapter 2 is met.

Table 3.2 Increment plan. Increment I is documented in Chapter 7 of this draft.

4
Data Flow Diagrams and Data Dictionary

4.1 Notation Used

The diagrams follow the conventional flow oriented notation. A circle is a process that transforms data. A rectangle is an external entity that lies outside the system boundary. An open ended pair of parallel lines is a data store. A labelled arrow is a data flow, and the label names the data in motion rather than the action being performed.

4.2 Level 0: Context Diagram

The context diagram fixes the system boundary. Aidly is shown as a single process numbered zero. Four human external entities interact with it, and ERSS 112 appears as a fifth external entity that receives an escalation handoff and never supplies data to the system. Placing 112 outside the boundary is the diagrammatic statement of the position taken in Chapter 1: the national service is escalated to, not integrated with and not replaced.

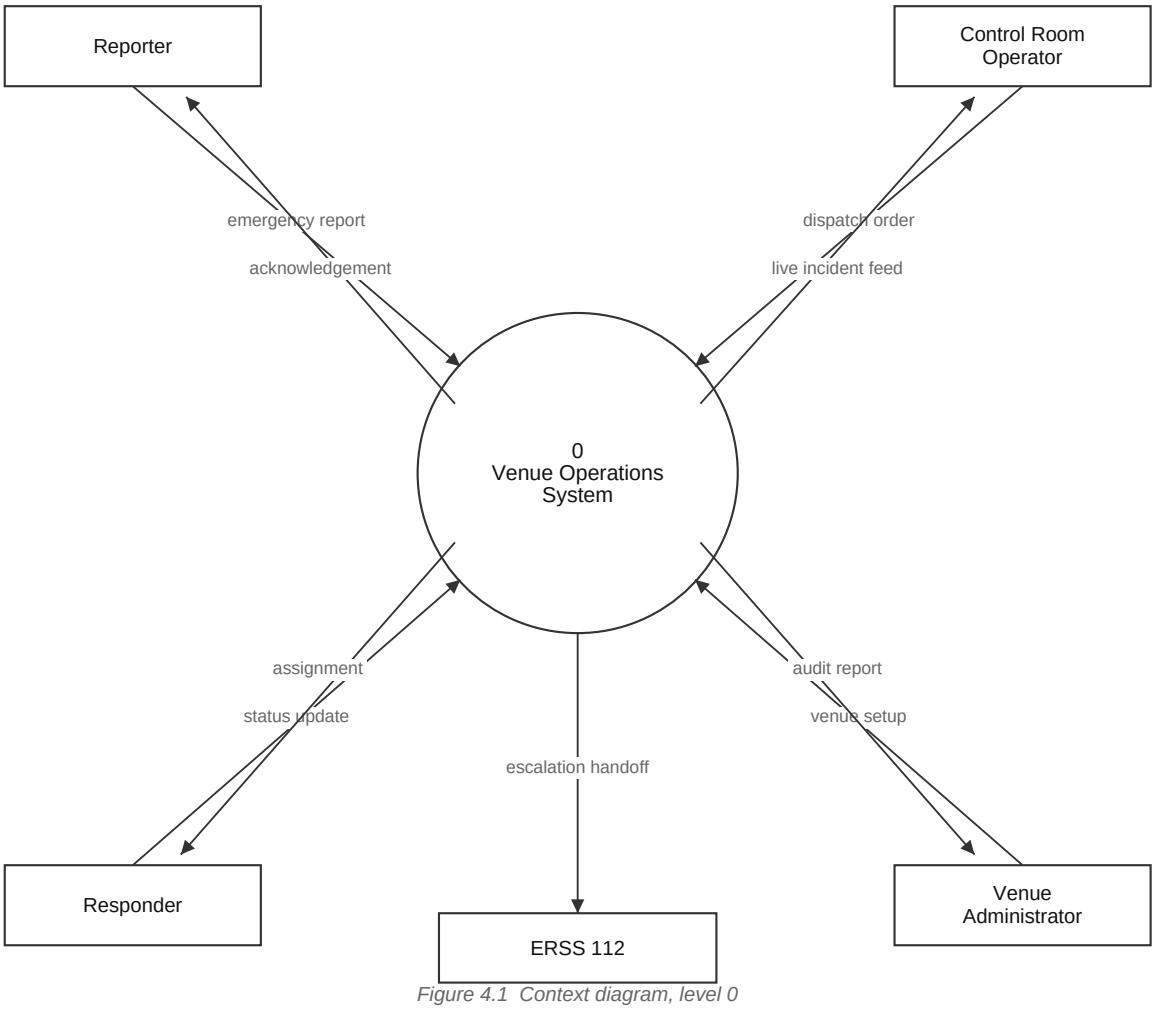


Figure 4.1 Context diagram, level 0

4.3 Level 1 Data Flow Diagram

The level 1 diagram decomposes process zero into the seven processes that correspond to the modules of the system. Data stores D1 to D7 are shown on the right and correspond one to one with the tables of the physical schema, so the diagram describes the system that is actually being built rather than a parallel abstraction.

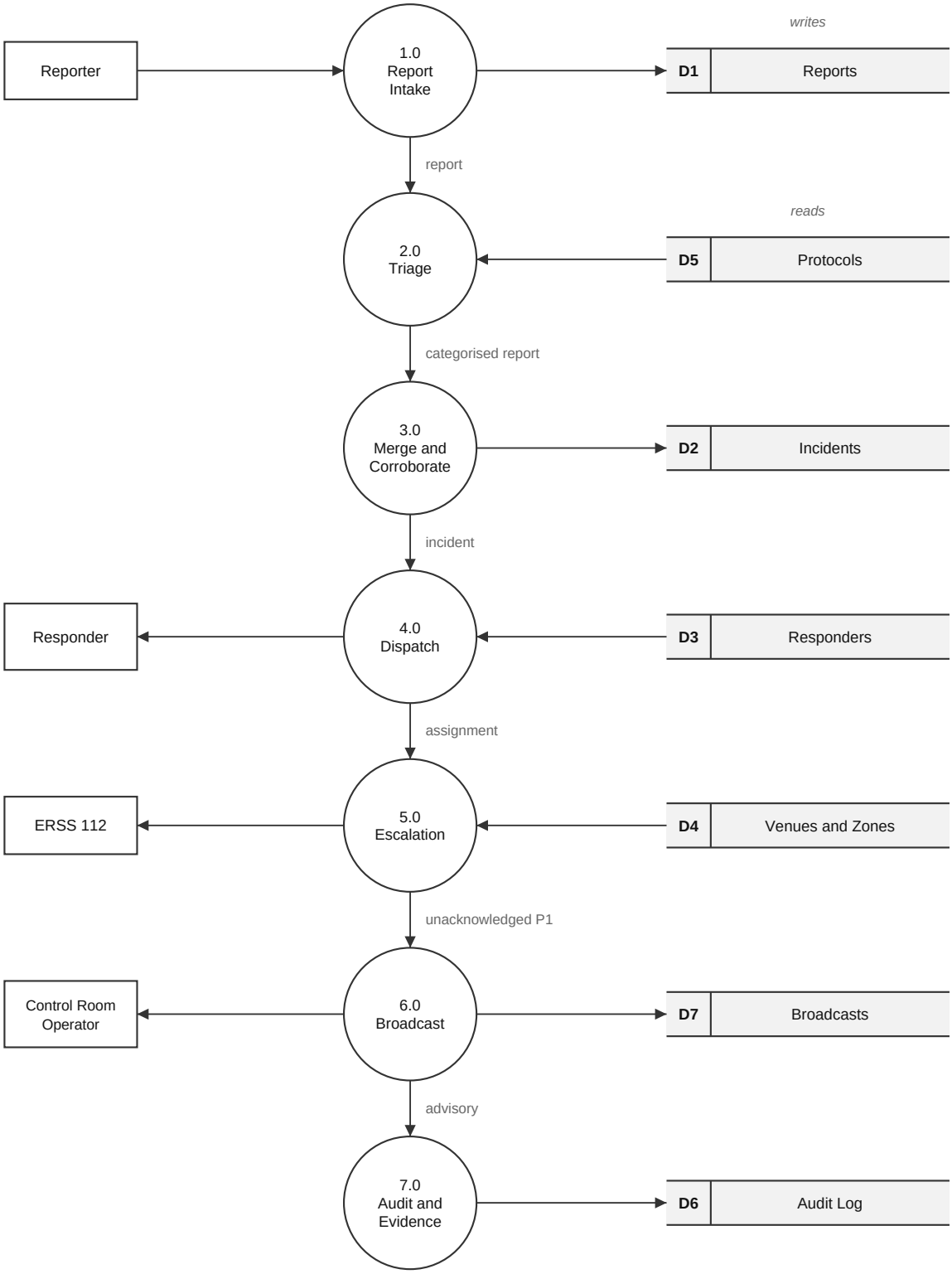


Figure 4.2 Data flow diagram, level 1

4.4 Data Dictionary

The dictionary defines every data flow and data store named in the diagrams. The notation is standard: = is composed of, + is and, **{ }** encloses a repeating group, **( )** encloses an optional item, and **[|]** encloses a choice of one alternative.

4.4.1 Data Flows

Name	Definition
emergency report	category + zone reference + timestamp + (description) + (photograph) + (callback number) + discreet flag
categorised report	emergency report + severity tier + triage method
incident	incident identifier + category + zone reference + severity tier + corroboration count + status + { report identifier }
assignment	incident identifier + responder identifier + issued at + acknowledgement deadline
acknowledgement	assignment identifier + responder identifier + acknowledged at
status update	incident identifier + [en route | on scene | resolved] + recorded at
unacknowledged P1	incident identifier + severity tier + elapsed seconds
escalation handoff	incident identifier + venue reference + zone name + category + severity tier + summary text
advisory	venue reference + (zone reference) + message text + issued at
live incident feed	{ incident + assignment status }
venue setup	venue name + { zone name + zone position + exit flag } + { responder record }
audit report	{ incident identifier + { event type + actor + occurred at } }

Table 4.1 Data flow definitions

4.4.2 Data Stores

Tag	Store	Definition
D1	Reports	report identifier + category + zone reference + (description) + (photograph reference) + (callback number) + discreet flag + received at + (incident identifier)
D2	Incidents	incident identifier + venue reference + zone reference + category + severity tier + (severity probability) + triage method + corroboration count + status + opened at + (resolved at)
D3	Responders	responder identifier + venue reference + code + name + role + position + availability flag + last seen at
D4	Venues and Zones	venue identifier + name + slug + centre position + { zone identifier + zone name + zone position + exit flag }
D5	Protocols	protocol identifier + category + title + body text + embedding vector

Tag	Store	Definition
D6	Audit Log	event identifier + incident identifier + event type + actor reference + occurred at + payload
D7	Broadcasts	broadcast identifier + venue reference + (zone reference) + message text + issued by + issued at

Table 4.2 Data store definitions

4.4.3 Key Data Elements

Element	Type and values
category	[fire | medical | crowd | accident | violence | harassment | other]
severity tier	[P1 | P2 | P3 | P4], where P1 is the most urgent
severity probability	Real number between 0 and 1, null until a trained classifier produces it
triage method	[rule | classifier | operator]
status	[triaging | verified | dispatched | en route | on scene | resolved]
role	[medic | fire | marshal | security]
user role	[operator | responder | admin]
discreet flag	Boolean. When true the client suppresses all sound, vibration and visible state change
corroboration count	Integer, at least 1, counting independent reports merged into the incident
position	Geographic point, latitude and longitude, SRID 4326

Table 4.3 Data element definitions

The separation of **severity probability** from **corroboration count** is deliberate. The first is a claim made by a model about one report. The second is evidence about how many people independently reported the same thing. Merging them into a single confidence figure would allow a weak model to masquerade as corroborated fact, which in an emergency tool could cause an operator to act wrongly. The probability is held as null until a trained classifier exists, because an absent number is honest and an invented one is not.

5
Use Case Diagram

5.1 Actors

Actor	Description
Reporter	Any attendee inside the venue. Not registered and not authenticated. Interacts only with the reporting screens.
Responder	A marshal, medic, fire warden or security member on duty at the venue. Authenticated. Receives and acts on assignments.
Control Room Operator	Supervises the venue. Authenticated. Sees every incident, dispatches responders, issues broadcasts and initiates escalation.
Venue Administrator	Configures the venue, its zones and its responder roster. Authenticated.
ERSS 112	External system actor. Receives an escalation handoff. Supplies no data to Aidly.

Table 5.1 Actors of the system

5.2 The Diagram

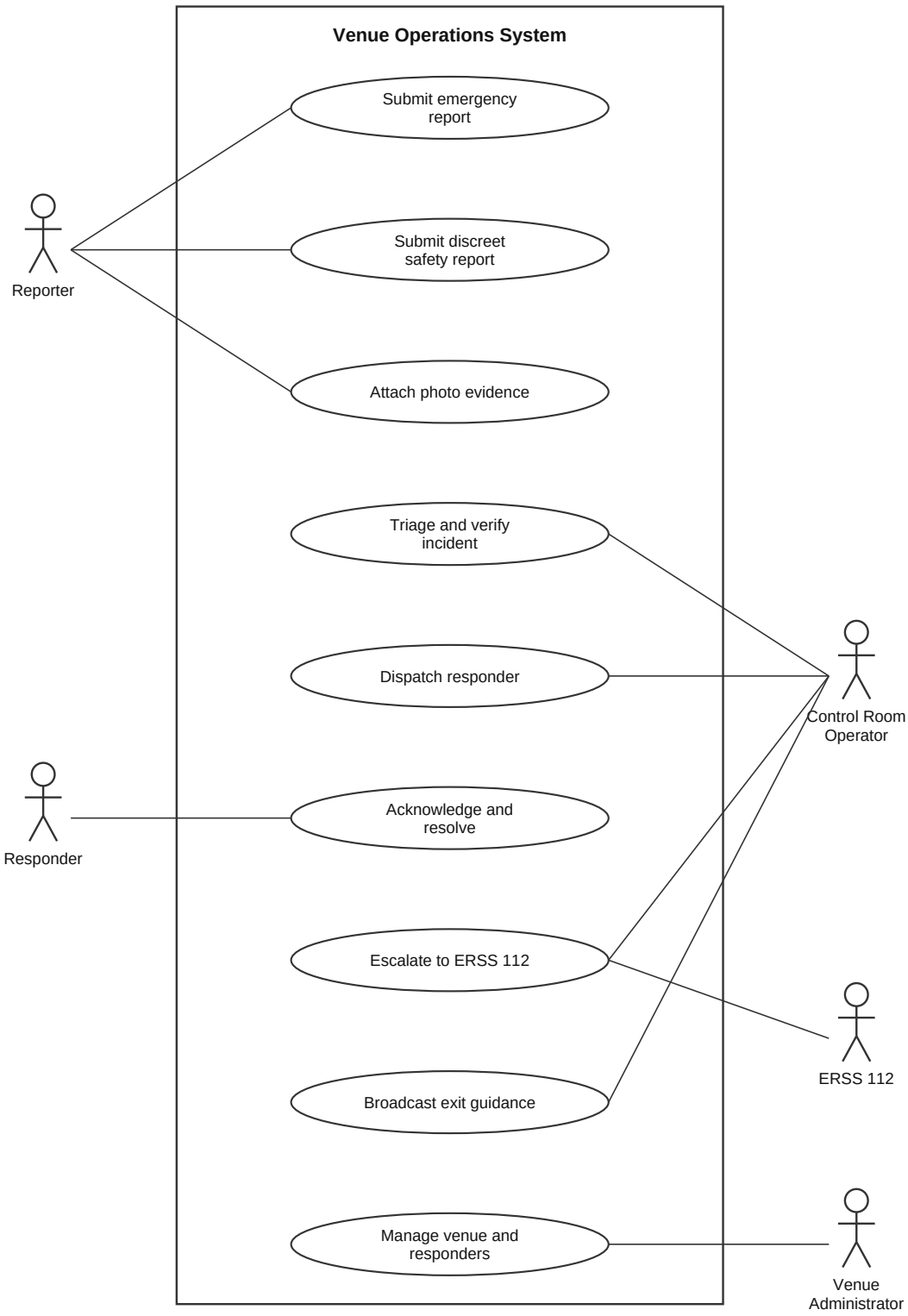


Figure 5.1 Use case diagram

5.3 Use Case Inventory

ID	Use case	Primary actor
UC 01	Submit emergency report	Reporter
UC 02	Dispatch responder	Control Room Operator
UC 03	Authenticate control room operator	Control Room Operator
UC 04	Submit discreet safety report	Reporter
UC 05	Attach photo evidence	Reporter
UC 06	Triage and verify incident	Control Room Operator
UC 07	Acknowledge and resolve assignment	Responder
UC 08	Escalate to ERSS 112	Control Room Operator
UC 09	Broadcast exit guidance	Control Room Operator
UC 10	Manage venue and responders	Venue Administrator

Table 5.2 Use case inventory. Templates for the three core cases UC 01, UC 02 and UC 03 follow in Chapter 6.

6
Use Case Templates

Templates are given for the three core requirements: the reporter path that begins every incident, the operator path that resolves it, and the authentication that guards both. The third is implemented in Chapter 7.

6.1 UC 01: Submit Emergency Report

Use case ID	UC 01
Name	Submit emergency report
Primary actor	Reporter, an attendee inside the venue
Requirements	FR 01, FR 02, FR 03, FR 04, FR 05
Description	An attendee who witnesses an emergency sends a categorised report to the venue control room in a single action, without registering.
Preconditions	The reporter has opened the Aidly reporting screen on a device with an active network connection. The venue is configured with at least one zone.
Postconditions	A report is persisted in store D1, resolved to a zone, and either creates a new incident in D2 or increments the corroboration count of an existing one.
Trigger	The reporter activates the reporting control on the landing screen.
Main flow	1. The reporter activates the reporting control. 2. The system presents the category chooser. 3. The reporter selects a category. 4. The system requests the device position. 5. The system resolves the position to the nearest zone and displays it. 6. The reporter optionally enters a description of at most 150 characters. 7. The reporter optionally attaches a photograph. 8. The reporter optionally enters a callback number. 9. The reporter submits the report. 10. The system persists the report, passes it to triage, and confirms receipt to the reporter.
Alternate flow A1	**Position unavailable.** At step 4 the browser refuses or fails to supply a position. The system displays a notice and presents a list of gates. The reporter selects the nearest gate and the flow resumes at step 6. The report is never blocked.
Alternate flow A2	**Duplicate event.** At step 10 an open incident exists with the same category and zone inside the configured time window. The system attaches the report to that incident and increments its corroboration count instead of opening a new incident.
Exception E1	**Submission fails.** The network request does not complete. The system retains the entered data, informs the reporter that the report has not been sent, and offers a retry. No silent failure is permitted.

Exception E2	**No zone configured.** The venue has no zones. The system accepts the report against the venue centre and flags it for operator attention.
Frequency	High. Expected to be the most invoked use case in the system.

6.2 UC 02: Dispatch Responder

Use case ID	UC 02
Name	Dispatch responder
Primary actor	Control Room Operator
Secondary actor	Responder
Requirements	FR 08, FR 09
Description	The operator assigns the most suitable available responder to an open incident and tracks the assignment to resolution.
Preconditions	The operator is authenticated as described in UC 03. An incident exists in D2 with status triaging or verified. At least one responder is recorded in D3.
Postconditions	The incident status becomes dispatched, an assignment record is written, the responder availability flag is cleared, and the action is recorded in the audit log D6.
Trigger	The operator selects an open incident and requests dispatch.
Main flow	1. The operator selects an open incident from the live feed. 2. The system displays the incident with its category, zone, severity tier and corroboration count. 3. The operator requests a dispatch recommendation. 4. The system determines the required role from the incident category. 5. The system queries D3 for available responders holding that role, ordered by distance from the incident zone. 6. The system presents the nearest matching responder. 7. The operator confirms the assignment. 8. The system records the assignment, marks the responder unavailable and notifies the responder device. 9. The responder acknowledges, and the incident status advances.
Alternate flow A1	**Attribute constrained request.** The incident category is harassment, or the reporter has requested an escort. At step 4 the system additionally requires the responder attribute recorded for that purpose, and the ordering at step 5 is restricted accordingly.
Alternate flow A2	**Operator override.** At step 7 the operator rejects the recommendation and selects a different responder. The system records both the recommendation and the override in D6.
Exception E1	**No matching responder available.** The query at step 5 returns nothing. The system does not fail silently. It widens the search to any available responder, marks the assignment as a fallback, and raises the incident for operator attention.
Exception E2	**Assignment not acknowledged.** The responder does not acknowledge within the configured interval. The escalation process is triggered, as described in FR 09.
Frequency	Moderate. Once per incident, occasionally more where an override or a reassignment occurs.

6.3 UC 03: Authenticate Control Room Operator

Use case ID	UC 03
Name	Authenticate control room operator
Primary actor	Control Room Operator
Requirements	FR 12, NFR 04, NFR 05
Description	An operator proves identity to the system and receives a time limited access token that authorises subsequent operational requests.
Preconditions	An account exists in the users store with an active flag set and a stored password hash. The application can reach the database.
Postconditions	On success the operator holds a signed access token carrying the account identifier, the role and the venue reference. On failure no token is issued and no information about the account is disclosed.
Trigger	The operator submits credentials to the login endpoint.
Main flow	1. The operator supplies an email address and a password. 2. The system normalises the address and looks up the account. 3. The system verifies the supplied password against the stored hash. 4. The system confirms that the account is active. 5. The system issues a signed access token carrying the identifier, the role and the venue reference. 6. The system returns the token, its lifetime and the safe account fields. 7. The client sends the token on every later request, and the system reloads the account to confirm it is still active.
Alternate flow A1	**Unknown account.** At step 2 no account matches. The system verifies the password against a fixed dummy hash so that the time taken does not differ, then follows the failure path.
Exception E1	**Wrong password.** Verification at step 3 fails. The system returns status 401 with the message *Incorrect email or password*. The same message is used for every failure cause so the response cannot be used to discover which accounts exist.
Exception E2	**Deactivated account.** The check at step 4 fails. The system responds exactly as for a wrong password.
Exception E3	**Malformed stored hash.** Verification raises rather than returning. The system treats the result as a failure and continues, so one corrupt row cannot disable the login endpoint.
Exception E4	**Database unreachable.** The system returns a server error and issues no token. It never falls back to a default or shared credential.
Frequency	Low. Once per operator per shift, plus token renewals.

7
Login Module Connection with Database

7.1 Overview

This chapter documents Increment I of the plan in Table 3.2: the login module, connected to a live PostgreSQL database. It implements UC 03 and satisfies FR 12, NFR 04 and NFR 05. The module is written in Python using the FastAPI framework and SQLAlchemy, and the code shown here is the code that produced the execution transcript in Section 7.7.

The module is deliberately split across four files. Hashing and token handling are isolated from the database, the database work is isolated from the HTTP layer, and the HTTP layer contains no security logic of its own. This is the functional independence described in the design unit of the syllabus, and it means the hashing scheme can be replaced without touching any endpoint.

File	Responsibility
app/db.py	Engine, session factory and the request scoped session dependency
app/core/security.py	Password hashing, password verification, token issue and token decode
app/services/auth.py	Account lookup and the authentication decision
app/routers/auth.py	The HTTP endpoints and the authorisation dependency

Table 7.1 Files comprising the login module

7.2 Database Schema

The users table replaces the single shared passcode used by the prototype. Each account carries its own hash, its own role and its own active flag.

```
class User(Base, PkMixin):
    """Replaces the shared passcode the prototype used."""
    __tablename__ = "users"

    email: Mapped[str] = mapped_column(String(255), unique=True, index=True)
    password_hash: Mapped[str] = mapped_column(String(255))
    display_name: Mapped[str] = mapped_column(String(120))
    role: Mapped[UserRole] = mapped_column(Enum(UserRole, name="user_role"))
    venue_id: Mapped[uuid.UUID | None] = mapped_column(
        ForeignKey("venues.id", ondelete="SET NULL"))
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)
```

Listing 7.1 The users table, defined in app/models.py

The email column is unique and indexed, which enforces one account per address and makes the lookup in Section 7.5 an index seek rather than a scan. There is no salt column: the chosen hashing scheme stores its salt inside the hash string itself.

7.3 Connection Layer

A single engine holds a pooled set of connections for the life of the process. Each request receives its own session, which is committed if the request succeeds and rolled back if it raises, and is closed in either case.

```
engine = create_engine(
    str(settings.database_url),
    pool_pre_ping=True,      # drop connections the database has already closed
    pool_size=10,
    max_overflow=20,
    echo=settings.debug,
)

SessionLocal = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)


def get_db() -> Generator[Session, None, None]:
    """FastAPI dependency. Commits on success, rolls back on any exception."""
    db = SessionLocal()
    try:
        yield db
        db.commit()
    except Exception:
        db.rollback()
        raise
    finally:
        db.close()
```

Listing 7.2 The connection layer, app/db.py

The connection string and the token signing secret are read from the environment and have no working defaults. If either is missing the application refuses to start rather than falling back to an insecure value, which is the failure the prototype exhibited with its shared operator passcode.

7.4 Password Hashing and Tokens

Passwords are stored only as bcrypt hashes. bcrypt generates a random salt for every password and applies a deliberately expensive derivation, so two identical passwords produce different stored values and a stolen table cannot be reversed by lookup.

```
BCRYPT_MAX_BYTES = 72


def _prepare(plain: str) -> bytes:
    """Encode and truncate on a character boundary, never mid codepoint."""
    raw = plain.encode("utf-8")
    if len(raw) <= BCRYPT_MAX_BYTES:
        return raw
    return raw[:BCRYPT_MAX_BYTES].decode("utf-8", "ignore").encode("utf-8")


def hash_password(plain: str) -> str:
    return bcrypt.hashpw(_prepare(plain), bcrypt.gensalt()).decode("ascii")


def verify_password(plain: str, hashed: str) -> bool:
    """Returns False on a malformed stored hash rather than raising, so one
```

```
    corrupt row cannot break the login endpoint."""
    try:
        return bcrypt.checkpw(_prepare(plain), hashed.encode("ascii"))
    except (ValueError, TypeError):
        return False
```

Listing 7.3 Password hashing, app/core/security.py

The 72 byte constant is not arbitrary. bcrypt ignores anything beyond the 72nd byte of a password. Left implicit, two different long passwords could silently share a hash, so the limit is applied in one visible place and the truncation respects character boundaries.

```
def create_access_token(subject: str, role: str,
                        venue_id: str | None = None) -> tuple[str, int]:
    expires_in = settings.access_token_minutes * 60
    now = datetime.now(timezone.utc)
    payload = {
        "sub": subject,
        "role": role,
        "venue_id": venue_id,
        "iat": now,
        "exp": now + timedelta(seconds=expires_in),
    }
    token = jwt.encode(payload, settings.jwt_secret,
                       algorithm=settings.jwt_algorithm)
    return token, expires_in
```

Listing 7.4 Access token issue, app/core/security.py

7.5 Authentication Service

The service is the only component that decides whether a credential is acceptable. Three rules govern it, and each answers a specific attack.

```
_DUMMY_HASH = "$2b$12$C6UzMDM.H6dfI/f/IKcEe.rJXQnGDDPFHOCkCcTLB0P9C7dQKrHQO"


def authenticate(db: Session, email: str, password: str) -> User | None:
    """Returns the user on success, None on every kind of failure."""
    normalised = email.strip().lower()
    user = db.execute(
        select(User).where(User.email == normalised)
    ).scalar_one_or_none()

    if user is None:
        verify_password(password, _DUMMY_HASH)
        return None

    if not verify_password(password, user.password_hash):
        return None

    if not user.is_active:
        return None

    return user
```

Listing 7.5 The authentication service, app/services/auth.py

- **One outcome for every failure.** The function returns None whether the account is unknown, the password is wrong, or the account is deactivated. The caller therefore cannot produce

different responses for different causes.

- **Constant work when the account is unknown.** A dummy hash is verified even though no user was found. Without this the unknown account path would return measurably faster and an attacker could enumerate valid addresses by timing alone. This directly satisfies NFR 05.
- **Normalisation before lookup.** The address is trimmed and lowered so that stored addresses are matched consistently.

7.6 The Login Endpoint

```
INVALID = HTTPException(
    status_code=status.HTTP_401_UNAUTHORIZED,
    detail="Incorrect email or password",
    headers={"WWW-Authenticate": "Bearer"},
)


@router.post("/login", response_model=TokenResponse)
def login(payload: LoginRequest, db: Session = Depends(get_db)) -> TokenResponse:
    user = authenticate(db, payload.email, payload.password)
    if user is None:
        raise INVALID

    token, expires_in = create_access_token(
        subject=str(user.id),
        role=user.role.value,
        venue_id=str(user.venue_id) if user.venue_id else None,
    )
    return TokenResponse(
        access_token=token,
        expires_in=expires_in,
        user=UserOut.model_validate(user),
    )
```

Listing 7.6 The login endpoint, app/routers/auth.py

The response is shaped by a schema that has no password field of any kind, so the hash cannot be returned by accident. Protected routes depend on a function that decodes the token and then reloads the account from the database, rather than trusting the claims alone. Deactivating an account therefore takes effect on the next request instead of whenever the token happens to expire.

7.7 Execution Transcript

The following is the unedited output of running the module against PostgreSQL 18 with the PostGIS and pgvector extensions enabled. The run creates the schema, seeds one operator account, performs a successful login, uses the returned token, and then exercises each rejection path.

```
============================================================
STEP 1  create schema in PostgreSQL
  tables: broadcasts, events, incidents, protocols, reports,
          responders, spatial_ref_sys, users, venues, zones
============================================================
STEP 2  seed one operator account
  stored id   : ef147f2e-0316-49bd-b7c5-afa30cf4dc88
  stored hash : $2b$12$qtZh22Y0qeBfLtpYMsqT5uR1Zs/Or ...
============================================================
```

```
STEP 3  POST /api/auth/login   correct password
  status: 200
  token : eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJlZjE ...
  type  : bearer  expires_in: 43200
  user  : Control Room One / operator
============================================================
STEP 4  GET /api/auth/me   with that token
  status: 200  email: control@arsd.example
============================================================
STEP 5  rejection cases
  wrong password: 401 Incorrect email or password
  unknown email : 401 Incorrect email or password
  no token      : 401
============================================================
```

Listing 7.7 Execution transcript of the login module

7.8 Outcome

Check	Expected	Observed
Schema creation	Nine application tables created	Passed
Password stored as a hash	bcrypt hash, never the plain value	Passed, prefix $2b$
Login with correct password	Status 200 with a signed token	Passed, lifetime 43200 seconds
Token accepted on a protected route	Status 200, correct account returned	Passed
Login with wrong password	Status 401, generic message	Passed
Login with unknown address	Status 401, identical message	Passed
Protected route without a token	Status 401	Passed
Tampered token	Rejected	Passed

Table 7.2 Verification of the login module

One defect was found and corrected during this increment. The project originally specified the passlib library for hashing. Current releases of bcrypt no longer expose the version attribute that passlib inspects, and the passlib backend probe raises on the 72 byte limit, so the module failed at the first call. The dependency was replaced with the bcrypt library used directly, which removed a layer rather than adding a workaround. Listing 7.3 shows the corrected code.

8
Conclusion and Next Steps

This draft covers the seven items required for the assignment. The problem has been stated and bounded, requirements have been identified through a working prototype and recorded as fourteen functional and eight nonfunctional requirements, the Incremental process model has been selected and justified against the four alternatives in the syllabus, the context and level 1 data flow diagrams have been drawn with a supporting data dictionary, the use case diagram and inventory have been produced, templates have been written for the three core use cases, and the login module has been implemented and verified against a live database.

Increment I of Table 3.2 is therefore complete. The work that follows, in order of dependency, is report intake with zone resolution, then triage and deduplication, then dispatch and escalation, and finally broadcast with the discreet reporting path and the audit record.

Three items are acknowledged as outstanding in this draft. The severity classifier described in FR 06 does not yet exist, and the severity probability field remains null by design until it does. The escalation interface to ERSS 112 is specified but not implemented, because no public programmatic interface to that service is published. The user interface shown in Chapter 2 is the throwaway prototype and not the delivered client.

References

1. Pressman, R. S., and Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach*. 9th ed. McGraw Hill.
2. Aggarwal, K. K., and Singh, Y. (2007). *Software Engineering*. 3rd ed. New Age International Publishers.
3. Jalote, P. (2005). *An Integrated Approach to Software Engineering*. 3rd ed. Narosa Publishing House.